UNED



Departamento de Sistemas de Comunicación y Control

Bases de datos clave-valor: Redis

Gestión / Almacenamiento de información no estructurada

Máster Universitario en Ingeniería y Ciencia de Datos

Andrés Duque Fernández

LECTURAS PARA PREPARAR EL TEMA

- Libro *Learning Redis*:
  - https://learning.oreilly.com/library/view/learning-redis/9781783980123/
- Libro *Redis 4.x Cookbook*:
  - https://learning.oreilly.com/library/view/redis-4x-cookbook/9781783988167/
- Documentación oficial Redis:
  - https://redis.io/
- Los enlaces de los libros son accesibles para usuarios autenticados en UNED

ÍNDICE

- Introducción
- Características
- Arquitectura
- Estructuras de datos
- Herramientas de desarrollo
- Modelado de datos
- Acceso programático a Redis
- Clustering en Redis
- Casos de uso
- Ejemplos de uso de Redis
- Herramientas gráficas

INTRODUCCIÓN

- Redis es una de las bases de datos NoSQL más populares, y seguramente la más utilizada dentro del segmento de las bases de datos clave-valor.
- Según el ranking de db-engines, Redis es la BBDD más popular entre las clave-valor, que incluye también a las bases de datos relacionales como Oracle, MySQL y SQL Server.

https://db-engines.com/en/ranking

INTRODUCCIÓN

- Motor de base de datos NoSQL basado en almacenamiento clave/valor.
  - Key("Ejemplo") → Value("Un ejemplo sencillo")

- Ofrece la posibilidad de almacenar la información en estructuras de datos diversas como listas, sets o hashes.

INTRODUCCIÓN

- Características más populares:
  - Velocidad: mantiene la información en memoria RAM (aunque también la puede mantener en disco).
  - Sencillez y flexibilidad.
- **Proyectos que necesiten bajos tiempos de respuesta**

INTRODUCCIÓN

- Redis se define usualmente como un servidor de estructuras de datos, aunque en sus inicios, haciendo honor a su nombre, fuese un simple diccionario remoto (REmote DIctionary Server).
- Tiene licencia BSD, está escrito en código C optimizado y admite numerosos lenguajes de desarrollo.
- Repositorio Redis:
  - https://github.com/antirez/redis/tree/4.0

CARACTERÍSTICAS

- **Velocidad elevada**: Los datos se encuentran en la memoria RAM, a diferencia de otros sistemas que los almacenan en disco.

- Al eliminar el acceso a esos discos, se evitan retrasos: se accede y se opera mucho más rápido.

CARACTERÍSTICAS

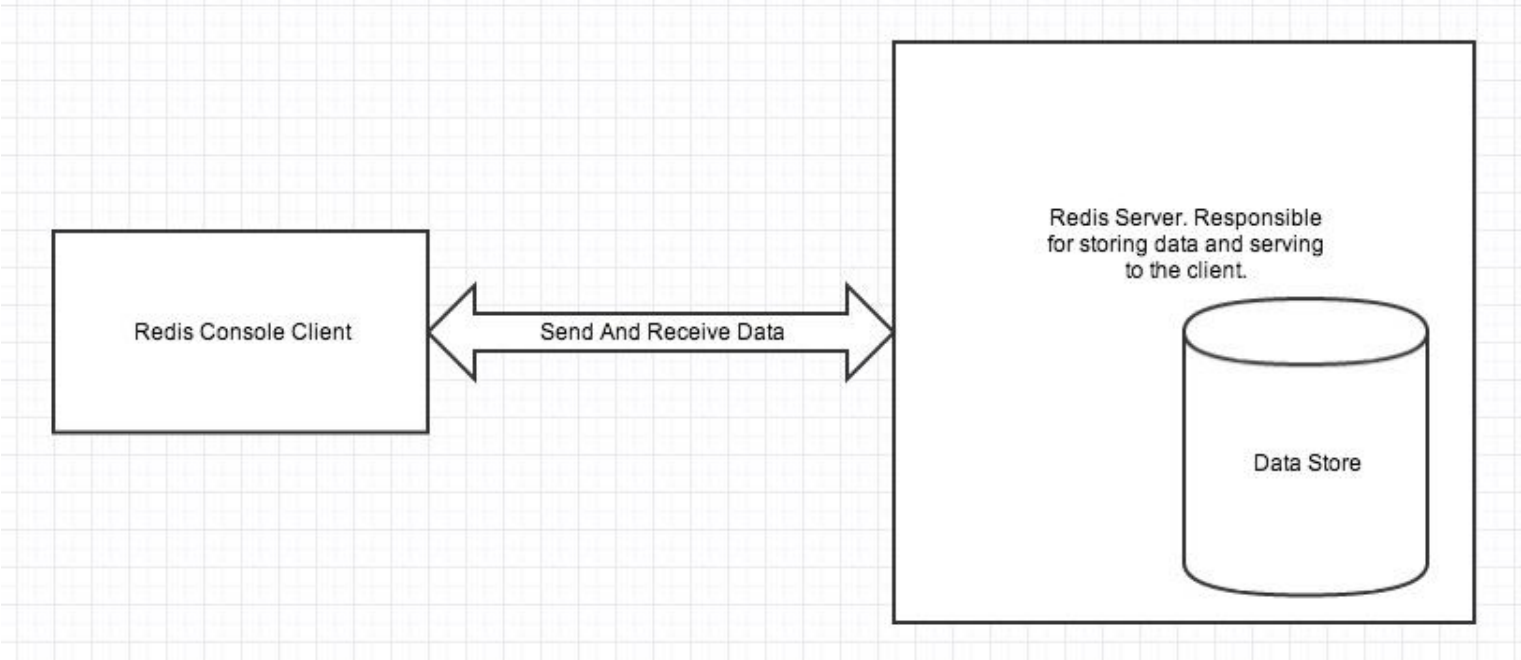
- **Estructuras de datos en memoria:**
  - Con Redis podemos guardar claves de diferentes tipos de datos y disponemos de un tamaño máximo de 512MB para una cadena, ya sea texto o datos binarios.
  - También admite listas de cadenas, conjuntos de cadenas sin orden, conjuntos clasificados por puntuación, hashes e HyperLogLogs.
  - Es decir, se puede almacenar en la memoria prácticamente cualquier tipo de dato.

CARACTERÍSTICAS

- **Replicación y persistencia:**
  - Arquitectura maestro-esclavo
  - Acepta replicación asíncrona: copia de datos en multitud de servidores esclavos.
  - Mejora de lecturas y recuperación cuando falla el servidor principal.

ARQUITECTURA

- **Instancia simple**

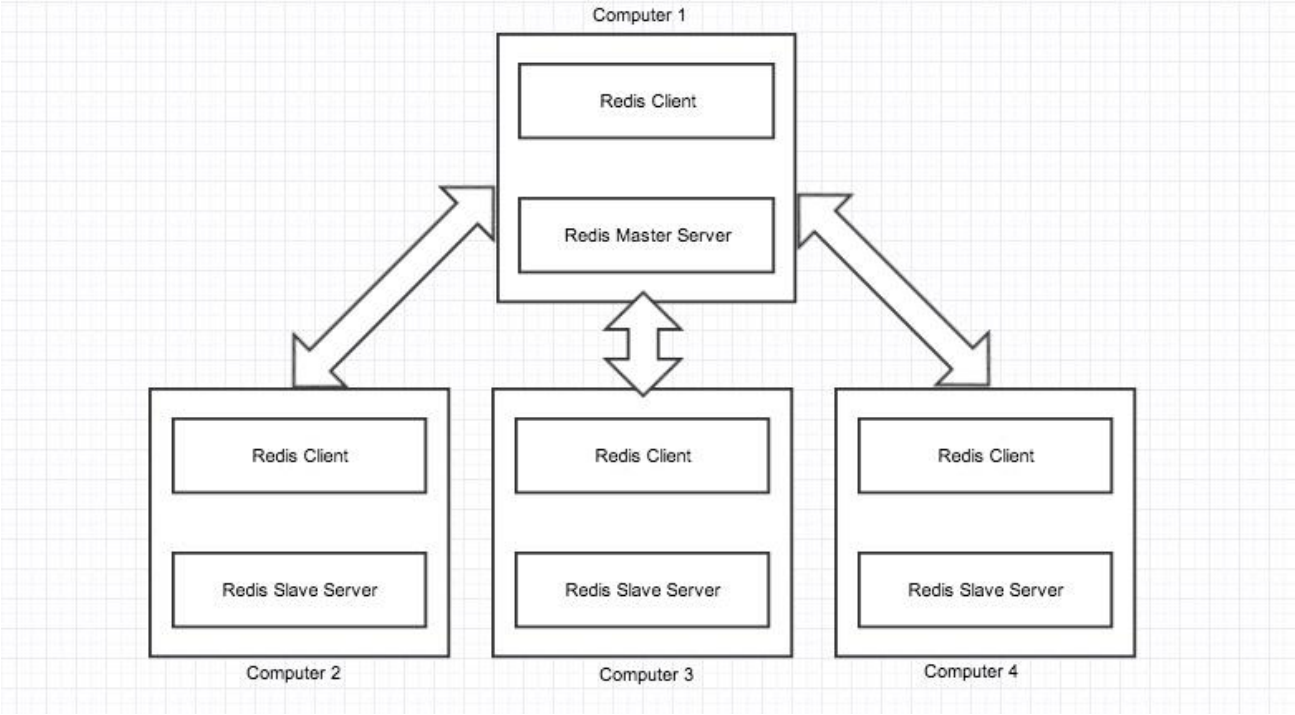


ARQUITECTURA

- **Sistema de replicación maestro-esclavo:**
  - Permite la escalabilidad en el servidor.
  - Un maestro puede tener múltiples esclavos, los cuáles pueden aceptar a su vez conexiones de otros esclavos.
  - No bloqueante por parte del maestro (el maestro sigue aceptando *queries* aunque los esclavos estén sincronizando). No bloqueante igualmente por parte de los esclavos.
  - Si el máster cae, el siguiente esclavo se convierte en máster.
  - Recuperación transparente en caso de fallo.

ARQUITECTURA

- **Sistema de replicación maestro-esclavo:**



ARQUITECTURA

- **Persistencia:**
  - RBD (*Redis Database Backup*): se copian mediante *snapshots* los datos en memoria a un almacenamiento permanente (disco), en un intervalo especificado.
  - AOF (*Append Only File*): se almacenan todas las operaciones de escritura recibidas en el servidor (causa archivos de *log* grandes).
  - Comando SAVE: se fuerza la creación de una *snapshot*.

ARQUITECTURA

- **Rendimiento:**
  - *Single-threaded* (otros sistemas crean un proceso o *fork* por cada petición nueva)
  - Para mejorar el rendimiento del CPU, se pueden iniciar múltiples instancias Redis en el mismo servidor.

ARQUITECTURA

- **Limitaciones:**
  - La limitación principal es la memoria (las estructuras de datos no pueden ser más grandes que la memoria). Si se supera la memoria se comienza a hacer *swapping*: afecta al rendimiento.
  - Opción *maxmemory* para limitar la memoria utilizada (a partir de ahí no se permiten nuevas escrituras, aunque sí lecturas).

ESTRUCTURAS DE DATOS

- **Comandos básicos:**
  - set <key> <value>: Se suele utilizar el separador ":" para crear la estructura de claves: *alumnos:uned*, *title:1*, *title:2*, *author:1*, *author:2*.
  - get <key>: Obtiene el valor de una clave (o *(nil)* si no existe).

ESTRUCTURAS DE DATOS

- **Strings:**
  - Son las estructuras más básicas disponibles.
  - Máximo de 512 MB.
  - Comandos comunes:
    - strlen <key>: cálculo de la longitud del valor
    - getrange <key> <start> <end>: subcadena en el rango indicado
    - append <key> <value>: añade <value> al final del valor existente (o crea uno nuevo si no hay ninguno).
    - incr <key>: Incrementa en uno el valor (si es posible)
    - decr <key>: Decrementa en uno el valor (si es posible)
    - incrby <key> <inc> : Incrementa en <inc> el valor (si es posible)
    - decrby <key> <dec>: Decrementa en <dec> el valor (si es posible)

ESTRUCTURAS DE DATOS

- **Hashes:**
  - Colección de pares clave-valor.
  - Especialmente adecuados para el almacenamiento de objetos.
  - hset <key> <field> <value> [<field> <value> …]
  - hget <key> <field>

ESTRUCTURAS DE DATOS

- **Hashes (otros comandos):**
  - hmget <key> <field> [<field>…]: obtiene el valor de varios campos.
  - hgetall <key>: obtiene todos los campos y sus valores.
  - hkeys <key>: obtiene sólo los campos.
  - hdel <key> <field> [<field>…]: elimina los campos especificados.

ESTRUCTURAS DE DATOS

- **Listas**
  - Permiten almacenar y manipular un conjunto de valores dada una clave concreta.
  - Se pueden añadir valores a la lista, recuperar valores y manipular valores de una posición concreta.
  - Las listas mantienen un orden y son eficientes al realizar operaciones basadas en su índice.
  - Los valores pueden ser cualquier tipo de datos.

ESTRUCTURAS DE DATOS

- **Listas (comandos comunes):**
  - lpush <key> <element> [<element>…]: inserta uno o varios elementos al inicio de la lista especificada con clave <key>.
  - lindex <key> <index>: Devuelve el elemento <index> de la lista (números negativos comienzan a contar por el final de la lista).
  - linsert <key> BEFORE | AFTER <pivot> <element>: Inserta un elemento antes o después de la posición especificada con <pivot>.
  - llen <key>: Devuelve la longitud de la lista.
  - lpop <key>: Devuelve el primer elemento de la lista y lo elimina de la misma.
  - Lrange <key> <start> <stop>: Devuelve el rango especificado de la lista.
  - Lrem <key> <count> <value>: Elimina los primeros (o los últimos si es un número negativo) <count> elementos de valor <value>. Si <count> vale 0, elimina todas las coincidencias.

ESTRUCTURAS DE DATOS

- **Conjuntos**
  - Almacenamiento de valores únicos.
  - No se mantiene el orden.
  - Operaciones eficientes (intersección, unión…).

ESTRUCTURAS DE DATOS

- **Conjuntos (comandos comunes):**
  - sadd <key> <member> [<member>…]: Añade uno o varios valores al conjunto.
  - scard <key>: Número de elementos del conjunto.
  - sunion <key> [<key>…]: Devuelve la unión de los conjuntos.
  - sinter <key> [<key>…]: Devuelve la intersección de los conjuntos.
  - sunionstore <dest><key>[<key>…]: Almacena la unión de los conjuntos en <dest>.
  - sinterstore <dest><key>[<key>…]: Almacena la intersección de los conjuntos en <dest>.

ESTRUCTURAS DE DATOS

- **Conjuntos ordenados**
  - Similares a los conjuntos.
  - Cada miembro está asociado con una puntuación, que se utiliza para representar el orden del conjunto de menor a mayor.

ESTRUCTURAS DE DATOS

- **Conjuntos ordenados (comandos comunes):**
  - zadd <key> <score> <member> [<score> <member>…]: Añade uno o varios valores al conjunto, con sus puntuaciones.
  - zcard <key>: Número de elementos del conjunto.
  - zcount <key> <min> <max>: Número de elementos del conjunto con puntuaciones entre <min> y <max>.
  - zrank <key> <member>: Devuelve la posición de <member> en el conjunto, ordenado de menor a mayor puntuación.
  - zrevrank <key> <member>: Devuelve la posición de <member> en el conjunto, ordenado de mayor a menor puntuación.
  - zrange <key><start><stop>: Devuelve los elementos en el rango especificado.

ESTRUCTURAS DE DATOS

- **Comando SORT:**
  - Probablemente uno de los comandos más complejos y potentes de Redis.
  - Permite emular las *queries* de SQL.
  - Opera sobre listas, conjuntos o conjuntos ordenados.
  - Por defecto, interpreta valores numéricos de coma flotante de doble precisión, y los ordena de forma ascendente.
  - Puede ser lento en conjuntos de datos grandes.
  - Definición:
    - SORT <key> [BY <pattern>] [LIMIT <offset> <count>] [GET <pattern> [GET <pattern> ...]] [ASC|DESC] [ALPHA] [STORE <destination>]

ESTRUCTURAS DE DATOS

- **Comando SORT:**
  - Uso simple: sort mylist / sort mylist ASC / sort mylist DESC
  - Uso de ALPHA: interpreta valores String para su ordenación lexicográfica.
  - Uso de LIMIT: Indica desde qué elemento mostrar (<offset>) y cuántos elementos mostrar (<count>).
  - Uso de STORE: Se indica que se desea guardar la salida de la ordenación en <destination>.

ESTRUCTURAS DE DATOS

- **Comando SORT:**
  - Uso de BY (claves externas): Se especifica un patrón, que incluye el carácter *, para indicar la clave que se quiere utilizar para ordenar.
  - Ejemplo:

```
sadd user:123:favourite_restaurants_ids 200 365 104 455 333
set rest_rating_200 4.3
set rest_rating_365 4.0
set rest_rating_104 4.8
set rest_rating_455 4.7
set rest_rating_333 4.6
sort "user:123:favourite_restaurants_ids" by rest_rating_* DESC
        1) 104
        2) 455
        3) 333
        4) 200
        5) 365
```

ESTRUCTURAS DE DATOS

- **Comando SORT:**
  - Uso de GET (claves externas): Se especifica uno o varios patrones, que incluyen el carácter *, para indicar los campos que se quieren mostrar
  - Ejemplo (siguiendo el ejemplo anterior):

```
set rest_name_200 "Ruby Tuesday"
set rest_name_365 "TGI Friday"
set rest_name_104 "Applebee's"
set rest_name_455 "Red Lobster"
set rest_name_333 "Boiling Crab"
sort "user:123:favourite_restaurants_ids" by rest_rating_* DESC get rest_name_*
        1) "Applebee's"
        2) "Red Lobster"
        3) "Boiling Crab"
        4) "Ruby Tuesday"
        5) "TGI Friday"
```

ESTRUCTURAS DE DATOS

- **Resto de comandos:**
  - Se puede encontrar la lista completa de comandos en https://redis.io/commands

HERRAMIENTAS DE DESARROLLO

- **HyperLogLog:**
  - Estructura de datos probabilística que se utiliza para ofrecer la cardinalidad de un conjunto (conteo de valores únicos).
  - Resultados aproximados, bajo uso de memoria (alrededor de 12 kilobytes, alrededor de 0,81% de tasa de error).
  - Útil para obtener el número de valores únicos en datasets muy grandes (normalmente una operación intensiva computacionalmente): usuarios de una web, conteo de palabras únicas en un libro, etc.

HERRAMIENTAS DE DESARROLLO

- **HyperLogLog (comandos):**
  - pfadd <key> <element> [<element>…]: Inserta elementos en un HLL. Devuelve 1 si su cardinalidad es susceptible de cambiar, 0 en caso contrario.
  - pfcount <key>[<key>…]: Conteo de elementos en un HLL. Con varias claves devuelve la suma de sus cardinalidades.
  - pfmerge <destkey> <sourcekey>[<sourcekey>…]: Almacena en <destkey> la unión de los HLLs almacenados en <sourcekey>.

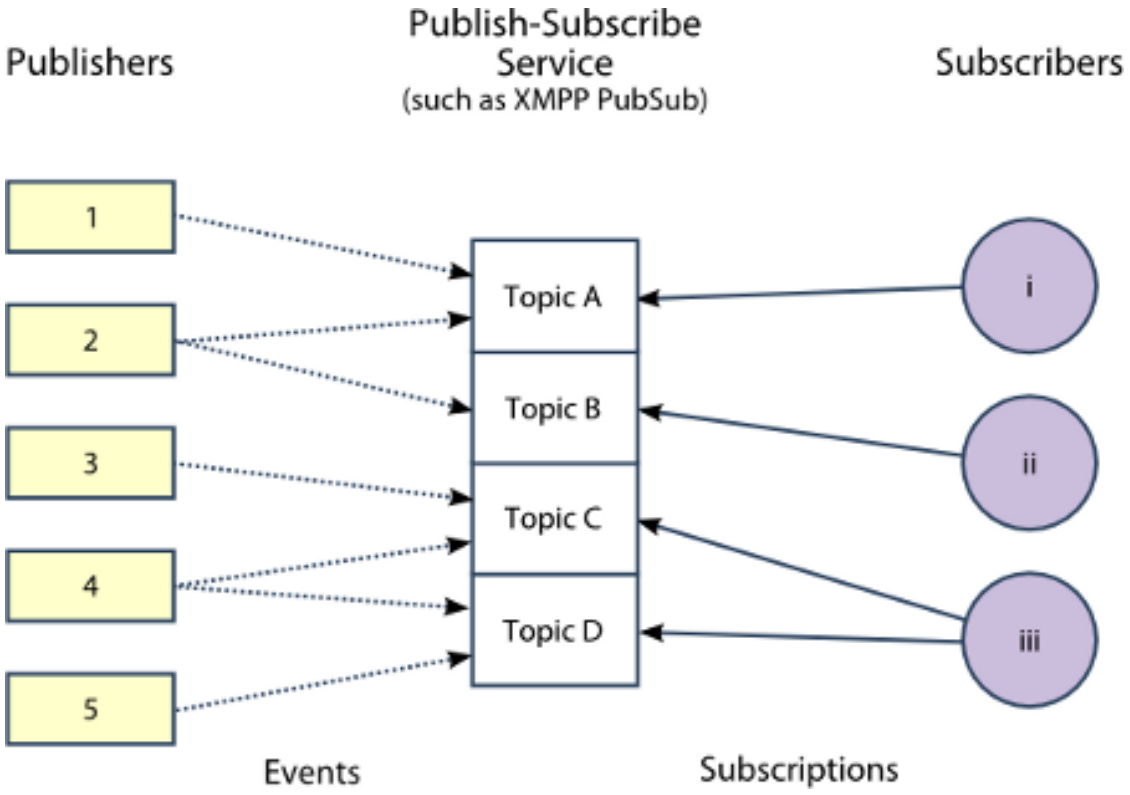
HERRAMIENTAS DE DESARROLLO

- **HyperLogLog (persistencia):**
  - HyperLogLog es básicamente un String de Redis.
  - Se puede obtener con GET y almacenarlo como String con SET.
  - Permite la copia y la persistencia de la estructura de datos.

HERRAMIENTAS DE DESARROLLO

- **Pub/Sub:**
  - Patrón de mensajería en el que varios Publishers envían mensajes a varios Subscribers, los cuáles escuchan mensajes en canales específicos a través del software de servidor.
  - El cliente Redis puede subscribirse a tantos canales como desee.
  - Uno de los patrones de cola de mensajes más utilizados, principalmente para sistemas de mensajería instantánea.

HERRAMIENTAS DE DESARROLLO



HERRAMIENTAS DE DESARROLLO

- **Pub/Sub (comandos):**
  - publish <channel><message>: Publicación de un mensaje en un canal.
  - subscribe <channel>[<channel>…]: Subscripción a uno o varios canales.
  - psubscribe <pattern>[<pattern>…]: Subscripción a canales siguiendo un patrón.
  - unsubscribe [<channel>[<channel>…]]: Anulación de la subscripción a uno o varios canales (o a todos si no se especifica ninguno).

HERRAMIENTAS DE DESARROLLO

- **Transacciones:**
  - Ejecución de un grupo de comandos en un solo paso.
  - Ejecución secuencial como una sola operación aislada (ninguna otra solicitud se servirá en medio de una transacción).
  - Operación atómica (se procesan todos los comandos o ninguno).

HERRAMIENTAS DE DESARROLLO

- **Transacciones (comandos):**
  - multi: Marca el inicio de una transacción. A partir de ahí, todos los comandos se pondrán en cola hasta que se ejecute la transacción.
  - exec: Ejecuta la transacción y vuelve al estado normal.
  - discard: Aborta la transacción (elimina la cola de comandos y vuelve al estado normal).

HERRAMIENTAS DE DESARROLLO

- **Sistema de claves TTL**
  - Determinadas claves especificas en Redis que indican un tiempo de vida determinado, tras el que se eliminan a sí mismas.
  - Útil para evitar llenar las bases de datos de datos no necesarios.

- **Contadores atómicos**
  - Aseguran que no se creen resultados incoherentes

HERRAMIENTAS DE DESARROLLO

- **LUA**
  - Lenguaje de Scripting liviano pero potente, estructurado e imperativo.
  - Gran versatilidad.

MODELADO DE DATOS

- **Modelado tradicional:**
  - Basado en tablas.
  - Datos predefinidos por columnas.
  - Modelo predefinido antes de la inserción de datos.
  - Consultas SQL.

MODELADO DE DATOS

- **Modelado Redis:**
  - No existe una predefinición de datos.
  - Se pueden recopilar las filas en una clave común.
  - La clave puede correr en torno al nombre del ítem:
    - shop:items:Gorgeus-Water-Mug
    - shop:items:Handcrafted-Trees-Mug

MODELADO DE DATOS

- **Modelado Redis:**
  - Aprovechando los hash, se pueden utilizar muchos campos no indexados con valores (cada campo ha de tener un valor).
  - Hash básico con 3 partes: KEY / FIELDS / VALUES:
    - hmset shop:items:Handcrafted-Trees-Mug artist "Martin Wessely" price 10.99 name "Handcrafted Trees Mug" description “Handcrafted Mug" manufacturer OHara-Group itemType mug productImg mug-400_002e1ecb8bd2.jpg

ACCESO PROGRAMÁTICO A REDIS

- Se pueden realizar conexiones a Redis con todo tipo de lenguajes de programación.
- Uno de los más sencillos y utilizados es Python: pip install redis

```
import redis
r = redis.Redis(host="localhost", port=6379, password="")
r.set('foo', 'bar')
value = r.get('foo')
print(value)
```

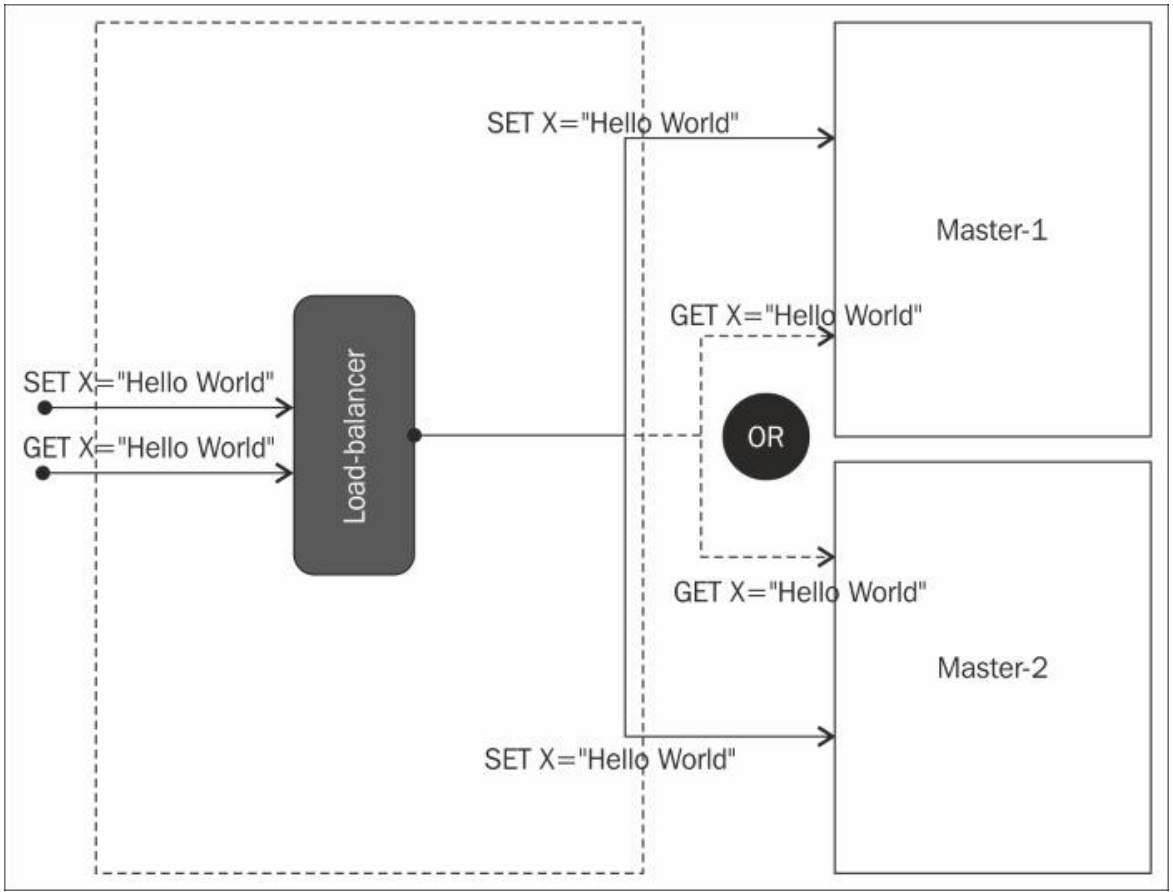
- Consultar en https://pypi.python.org/pypi/redis

CLUSTERING EN REDIS

- Conjunto de ordenadores conectados trabajando conjuntamente de tal forma que, en muchos aspectos, funcionan como un sistema único.
- Motivaciones:
  - Escalabilidad
  - Rendimiento
  - Disponibilidad
- Mayor esfuerzo en:
  - Gestión
  - Mantenimiento
  - Seguridad

CLUSTERING EN REDIS

- **Patrón Maestro-Maestro:**



CLUSTERING EN REDIS

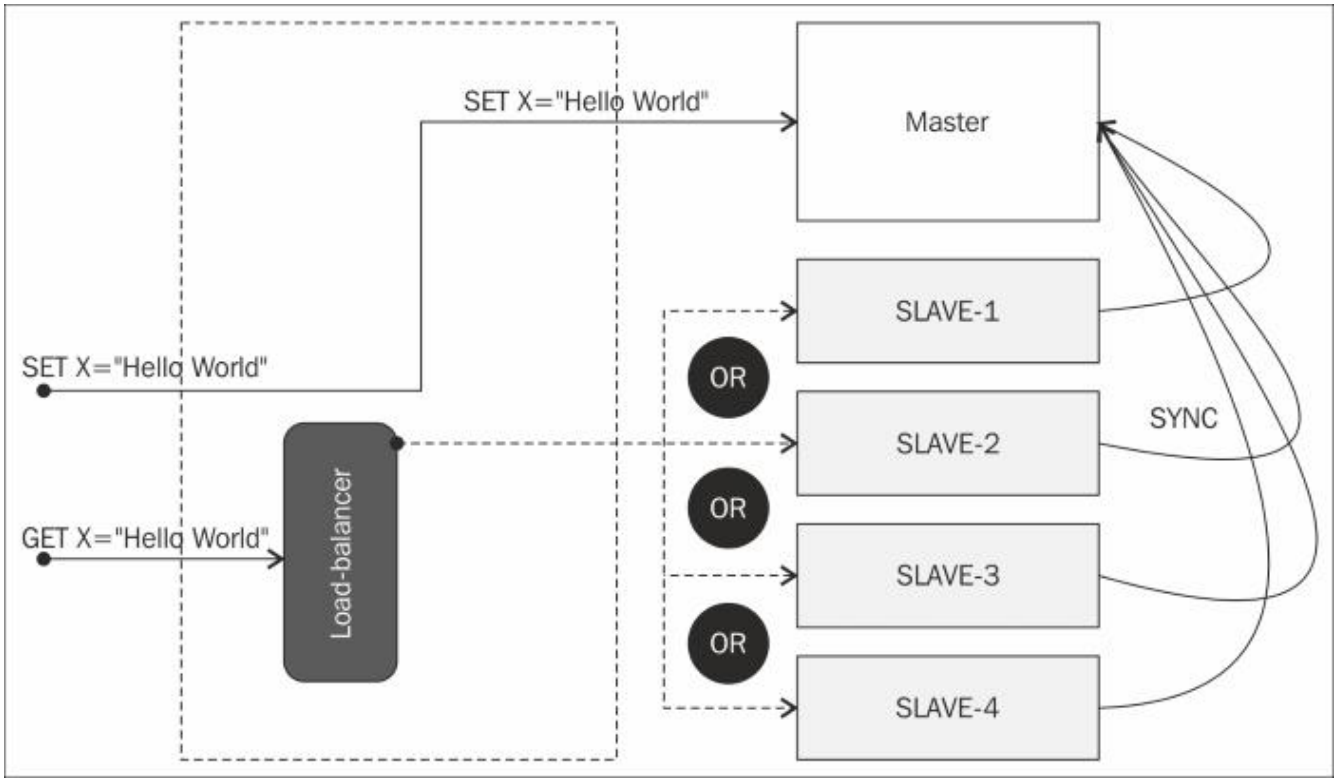
- **Patrón Maestro-Maestro:**
  - Alto rendimiento (necesidad de un balanceo de carga previo, no proporcionado por Redis).
  - Alta disponibilidad, dependiente del número de nodos maestros (se mantiene el estado en todos los nodos).
  - Escalabilidad comprometida por el proceso de añadido de un nuevo nodo: alto impacto en la disponibilidad.
  - Gestión: esfuerzo elevado, ya que no existe un mecanismo ofrecido por Redis para implementar este patrón.
  - Baja seguridad: contraseñas en texto plano, no hay soporte para encriptación. Se puede realizar ofuscación de comandos.

CLUSTERING EN REDIS

- **Patrón Maestro-Maestro. Desventajas:**
  - Paradas planeadas (caída o añadido de nodos).
  - Escalado horizontal complejo (necesidad de una planificación detallada).

CLUSTERING EN REDIS

- **Patrón Maestro-Esclavo:**



CLUSTERING EN REDIS

- **Patrón Maestro-Esclavo:**
  - Lecturas frecuentes (se realizan en los esclavos, balanceando la carga), escrituras no tan frecuentes (se realizan en el maestro).
  - Datos que puedan caber en el hardware ofrecido por el maestro.
  - Escalabilidad horizontal.
  - Aumento de latencia al replicar datos en esclavos.
  - Especialmente utilizado en la utilización de Redis para la implementación de cachés.

CLUSTERING EN REDIS

- **Patrón Maestro-Esclavo:**
  - Alto rendimiento (el balanceo de carga ha de ser proporcionado por otra aplicación).
  - Disponibilidad sencilla en los esclavos (si cae uno hay más).
  - Disponibilidad compleja en el máster (se detienen las escrituras):
    - Cola de mensajes previa al máster.
    - **Sentinel**: Mecanismo proporcionado por Redis.
  - Escalabilidad (sólo afecta al rendimiento): los nuevos esclavos han de presentar capacidades hardware similares al maestro, y han de ser registrados en el balanceador de carga previo.
  - Gestión: Esfuerzo menor (mecanismo proporcionado por Redis, por lo que la replicación y carga de datos en los esclavos están cubiertas). Gestión del máster y del balanceador de carga/adaptadores de clientes.
  - Seguridad: Similar al patrón Maestro-Maestro.

CLUSTERING EN REDIS

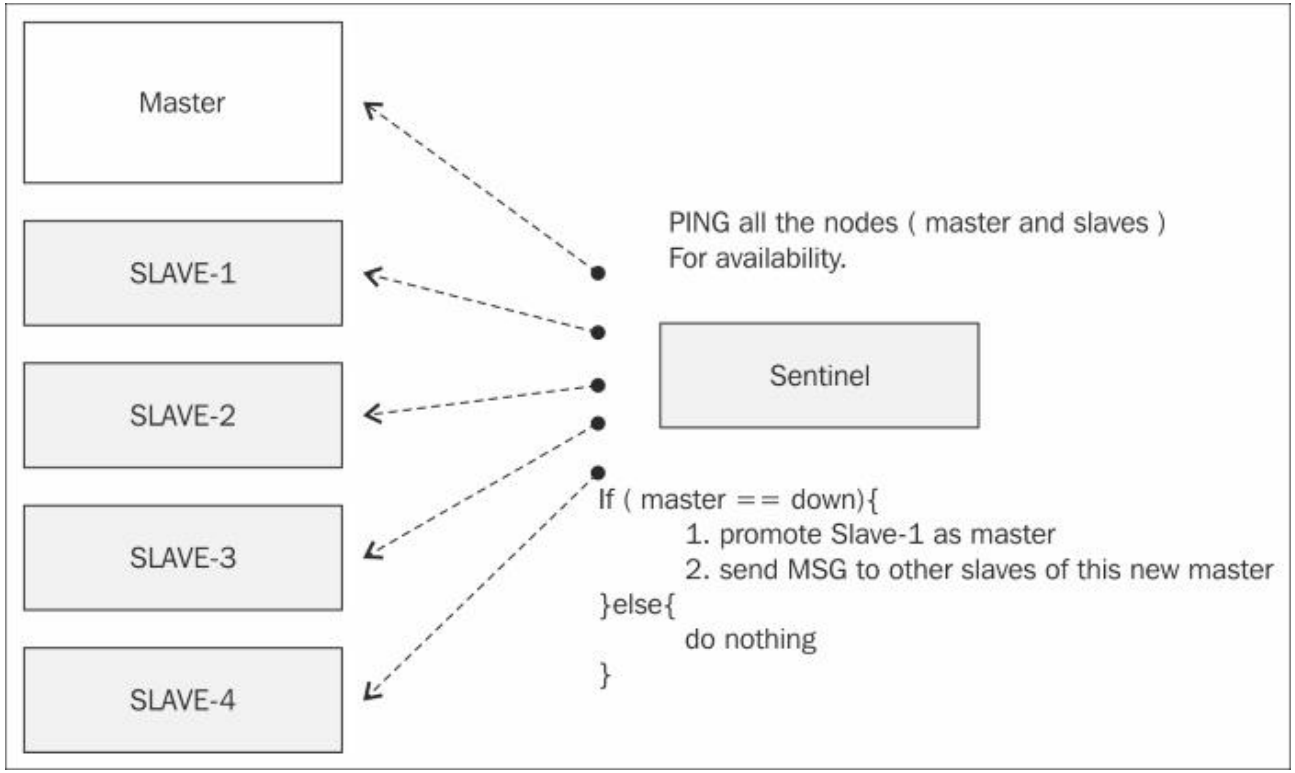
- **Patrón Maestro-Esclavo. Desventajas:**
  - Uno de los mayores problemas es la cantidad de datos a gestionar.
    - Capacidad hardware y escalabilidad vertical del maestro.
    - Capacidad hardware de los esclavos.
  - Latencia en la copia de datos: datos caducados (*stale data*).

CLUSTERING EN REDIS

- **Redis Sentinel**:
  - Nodo independiente de Redis que gestiona la tolerancia a fallos.
  - Mantiene un registro sobre el maestro y los esclavos.
  - Si el maestro cae, promociona a un esclavo para que se convierta en maestro.
  - Lo normal es utilizar varios nodos Sentinel:
    - Se controla la eventualidad de que un nodo Sentinel caiga.
    - Se promociona a un esclavo si varios de los nodos Sentinel han detectado la caída del maestro.
    - Proceso automático y transparente al usuario

CLUSTERING EN REDIS

- **Redis Sentinel**:



CLUSTERING EN REDIS

- **Redis Sentinel. Opciones**:
  - Uso: sentinel monitor <master-group-name> <ip> <port> <quorum>
    - <master-group-name>: Nombre del maestro
    - <ip> <port>: IP y puerto del maestro
    - <quorum>: Número de Sentinels que han de detectar la caída del maestro para iniciar una recuperación (si es posible).
    - Para esa recuperación, la mayoría de Sentinels tienen que estar activos (*reachables*) y votar al Sentinel líder que guiará la recuperación.

CLUSTERING EN REDIS

- **Redis Sentinel. Opciones**:
  - <down-after-miliseconds>: Tiempo mínimo en milisegundos para que el Sentinel comience a considerar que el nodo ha caído.
  - <parallel-syncs>: Número de réplicas que se pueden reconfigurar al mismo tiempo durante una recuperación (operación de gestión para atender al nuevo maestro). Normalmente se le da valor 1 para evitar problemas de latencia.
  - Documentación de Sentinel: https://redis.io/topics/sentinel

CASOS DE USO

- **Cachés:**
  - **Páginas web:**
    - Se puede utilizar como caché de páginas HTML o fragmentos de éstas.
    - Se acelera el acceso a las mismas.
    - Reducción de la carga en servidores Web y de aplicaciones.
    - Ahorro económico (HW y licencias SW).
  - **Bases de datos:**
    - Resultados de consultas muy frecuentes, con información poco cambiante o cuya actualización no es crítica.
    - Rebaja la carga de bases de datos operacionales.

CASOS DE USO

- **Almacenamiento de sesiones de usuario:**
  - Sesiones de muy rápido acceso.
  - Identificador de sesión + información de acceso a la misma.
  - Reduce el tiempo de latencia y acceso a otras bases de datos.
  - Se puede asociar un tiempo de expiración a cada clave:
    - expire <key> <seconds>

CASOS DE USO

- **Almacenamiento de carritos de la compra:**
  - Similar al almacén de sesiones de usuarios.
  - Artículos contenidos en la cesta del usuario + información de acceso.
  - Reducción del tiempo de latencia.

CASOS DE USO

- **Contadores y estadísticas:**
  - Manejo en tiempo real.
  - Soporte para gestión concurrente y atómica.
  - Ejemplos:
    - Visualizaciones de un producto.
    - Votaciones.
    - Acceso a un recurso.

CASOS DE USO

- **Listas de elementos recientes:**
  - Muestra de últimas actualizaciones de un elemento realizadas por los usuarios.
  - Normalmente costoso para BBDD operacionales (sobre todo con alto volumen de información).
  - En Redis: independencia del volumen.
  - Ejemplos:
    - Últimos comentarios sobre un post.
    - Últimas imágenes subidas.
    - Artículos de un catálogo vistos recientemente.

CASOS DE USO

- **Base de datos principal:**
  - Para casos determinados, como microservicios.
  - Aprovecha su potencia de modelado.
  - Soluciones especializadas, simples de implementar y mantener.
  - Alto rendimiento.

EJEMPLOS DE USO DE REDIS

- **Twitter**:
  - Mantenimiento del *timeline* de los usuarios.
  - Actualización en el menor tiempo posible.
  - Operación crítica.
- **Pinterest**:
  - Mantenimiento de información de usuarios y tablones que sigue cada uno.
- **Trello**:
  - Mantenimiento de información efímera.
  - Compartida entre todos sus servidores.

HERRAMIENTAS GRÁFICAS

- **Terminal:** redis-cli
- **FastoRedis:** http://fastoredis.com
- **Redis Desktop Manager:** https://redisdesktop.com

LECTURAS PARA PREPARAR EL TEMA

- Libro *Learning Redis*:
  - https://learning.oreilly.com/library/view/learning-redis/9781783980123/
- Libro *Redis 4.x Cookbook*:
  - https://learning.oreilly.com/library/view/redis-4x-cookbook/9781783988167/
- Documentación oficial Redis:
  - https://redis.io/
- Los enlaces de los libros son accesibles para usuarios autenticados en UNED